

Reconocimiento de Dígitos con Red Neuronal y Cámara USB

Maryom Alejandra Vicente González

Carné: 202202918

Universidad de San Carlos de Guatemala

Resumen—Se entrenó una red neuronal con MNIST y luego se utilizó el modelo en Python para realizar predicciones con una cámara USB.

I. INTRODUCCIÓN

Se trabajó con MNIST para reconocer números escritos a mano. El procedimiento se realizó en Python tomando como base el ejemplo de Google Colab y luego se probó el modelo con una cámara USB.

II. OBJETIVOS

- Entrenar una red neuronal con MNIST.
- Realizar el procedimiento en Python.
- Probar el modelo con una cámara USB.

III. CÓDIGO EN PYTHON

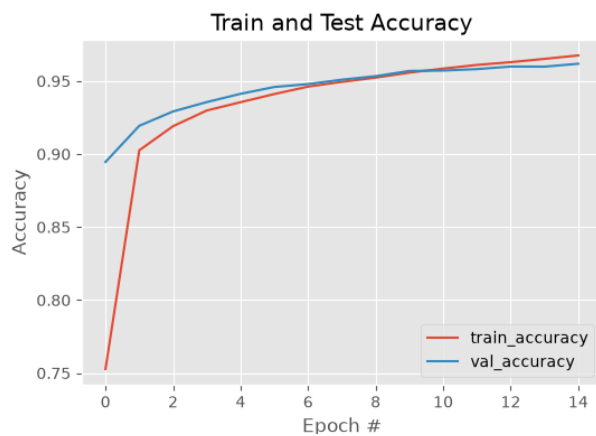
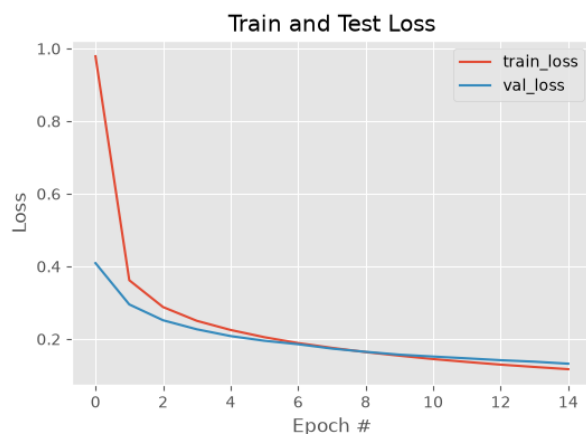
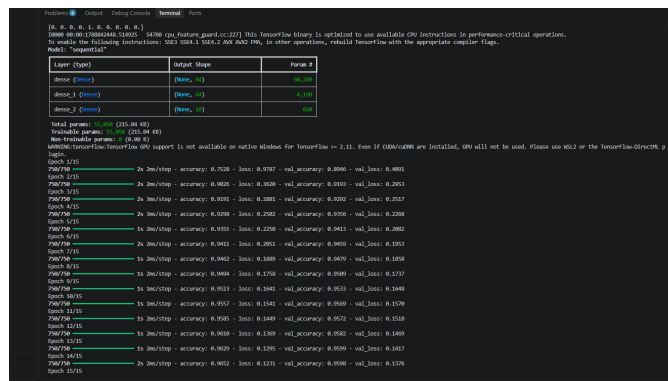
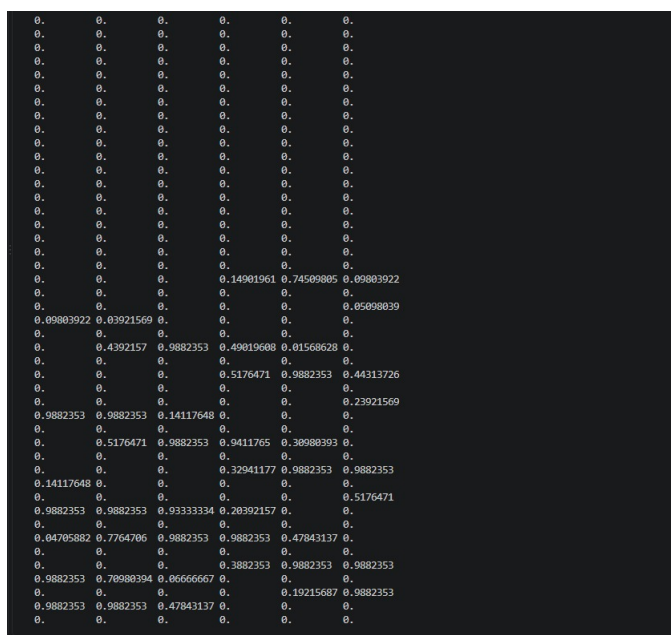
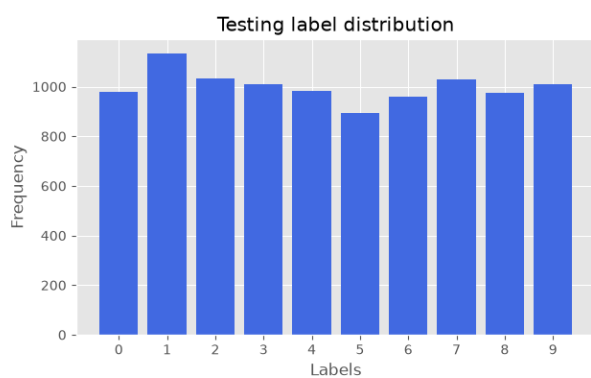
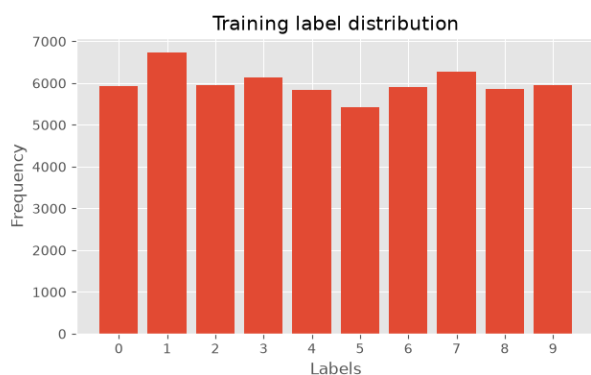
En las siguientes imágenes se muestra el código utilizado para cargar los datos, prepararlos, entrenar la red neuronal y guardar el modelo.

```
C:\Users\maryo\Desktop>python> T60k.py > ...
1  import sys
2  import os
3
4  # Python 3.13
5  python313 - r"C:\Users\maryo\AppData\Local\Programs\Python\Python313\python.exe"
6  if sys.version_info[:2] != (3, 13):
7      os.execv(python313, [python313] + sys.argv)
8
9  # Librerías
10 import numpy as np
11 import matplotlib.pyplot as plt
12 import tensorflow as tf
13 from tensorflow.keras.utils import to_categorical
14 from tensorflow.keras import Sequential
15 from tensorflow.keras.layers import Input, Dense
16 from tensorflow.keras.optimizers import Adam
17
18 plt.style.use("ggplot")
19
20 # Cargar MNIST
21 (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
22
23 # Dimensiones
24 print("Datos de entrenamiento:", X_train.shape, y_train.shape)
25 print("Datos de prueba:", X_test.shape, y_test.shape)
26
27 # Pruebas
28 print(X_train[8])
29 print(y_train[8])
30 print(X_train[20])
31 print(y_train[20])
32
33 # Gráfico entrenamiento
34 values_train, count_train = np.unique(y_train, return_counts=True)
35
36 plt.figure(figsize=(7, 4))
37 plt.bar(values_train, count_train)
38 plt.title("Training label distribution")
39 plt.xlabel("Labels")
40 plt.ylabel("Frequency")
41 plt.xticks(values_train)
42 plt.show()
43
```

Figura 1. Código en Python, parte 1.

```
44 # Gráfico prueba
45 values_test, count_test = np.unique(y_test, return_counts=True)
46
47 plt.figure(figsize=(7, 4))
48 plt.bar(values_test, count_test, color="royalblue")
49 plt.title("Testing label distribution")
50 plt.xlabel("Labels")
51 plt.ylabel("Frequency")
52 plt.xticks(values_test)
53 plt.show()
54
55 # Preparar imágenes
56 train_images = X_train.reshape((60000, 28 * 28))
57 train_images = train_images.astype("float32") / 255
58
59 test_images = X_test.reshape((10000, 28 * 28))
60 test_images = test_images.astype("float32") / 255
61
62 # Pruebas
63 print(train_images.shape)
64 print(train_images[20])
65
66 # Preparar etiquetas
67 train_labels = to_categorical(y_train)
68 test_labels = to_categorical(y_test)
69
70 # Prueba
71 print(train_labels[20])
72
73 # Crear modelo
74 model = Sequential([
75     Input(shape=(28 * 28,)),
76     Dense(64, activation="relu"),
77     Dense(64, activation="relu"),
78     Dense(10, activation="softmax")
79 ])
80
81 # Resumen
82 model.summary()
83
84 # Compilar
85 model.compile(
86     optimizer=Adam(learning_rate=0.0001),
87     loss="categorical_crossentropy",
88     metrics=["accuracy"]
89 )
90
```

Figura 2. Continuación del código en Python.



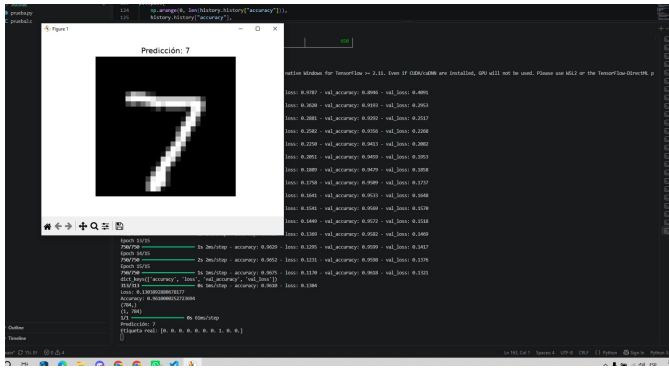


Figura 14. Resultado obtenido en Python.

V. CÓDIGO DE CÁMARA

Después se utilizó OpenCV para abrir una cámara USB y enviar la imagen al modelo entrenado.

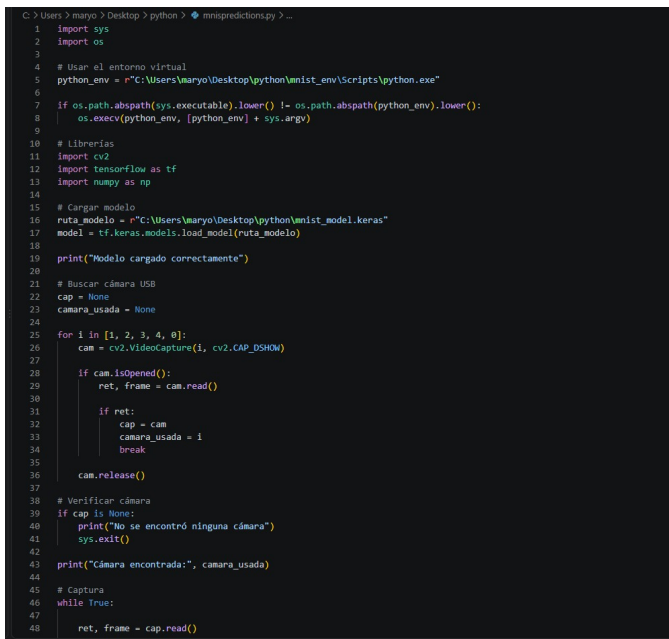


Figura 15. Código de cámara, parte 1.

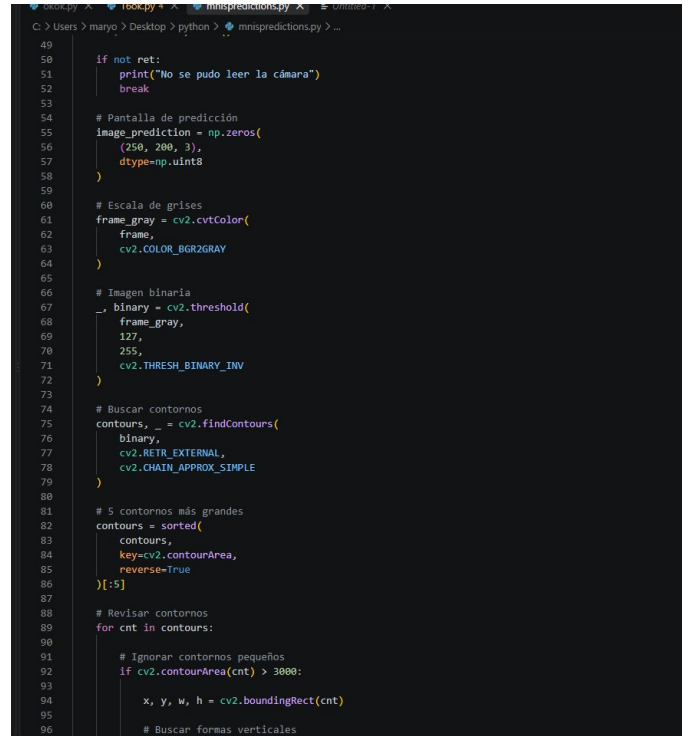


Figura 16. Continuación del código de cámara.

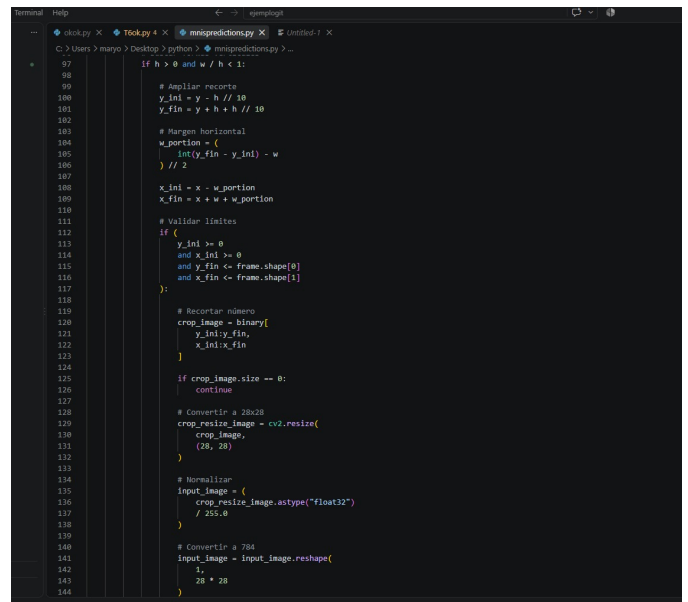


Figura 17. Continuación del código de cámara.

```

C:\Users\marco > Desktop > python > mnispredictions.py ...
144
145
146     # Predicción
147     prediction = model.predict(
148         input_image,
149         verbose=0
150     )
151
152     # Clase predicha
153     predicted_class = np.argmax(
154         prediction,
155         axis=1
156     )[0]
157
158     # Porcentaje
159     porcentaje = (
160         prediction[0][predicted_class]
161         * 100
162     )
163
164     print(
165         "Predicción:",
166         predicted_class,
167         f"- {porcentaje:.1f}%"
168     )
169
170     # Rectángulo original
171     cv2.rectangle(
172         frame,
173         (x, y),
174         (x + w, y + h),
175         (255, 0, 0),
176         2
177     )
178
179     # Rectángulo ampliado
180     cv2.rectangle(
181         frame,
182         (x_ini, y_ini),
183         (x_fin, y_fin),
184         (0, 255, 0),
185         2
186     )
187
188     # Mostrar recorte
189     cv2.imshow(
190         "Numero detectado",
191         crop_image

```

Figura 18. Continuación del código de cámara.

```

C:\Users\marco > Desktop > python > mnispredictions.py ...
191
192
193     crop_image
194
195     cv2.imshow(
196         "Numero 28x28",
197         crop_resize_image
198     )
199
200     # Mostrar porcentaje
201     cv2.putText(
202         image_prediction,
203         f"Predicción: {porcentaje:.1f}%",
204         (5, 25),
205         cv2.FONT_HERSHEY_SIMPLEX,
206         0.5,
207         (0, 255, 255),
208         1
209     )
210
211     # Mostrar número
212     cv2.putText(
213         image_prediction,
214         str(predicted_class),
215         (35, 210),
216         cv2.FONT_HERSHEY_SIMPLEX,
217         6,
218         (0, 255, 255),
219         3
220     )
221
222     # Mostrar ventanas
223     cv2.imshow("Camara USB", frame)
224     cv2.imshow("Predicción", image_prediction)
225     cv2.imshow("Binaria", binary)
226
227     # ESC para salir
228     if cv2.waitKey(1) & 0xFF == 27:
229         break
230
231     # Cerrar
232     cap.release()
233     cv2.destroyAllWindows()
234

```

Figura 19. Continuación del código de cámara.

VI. RESULTADOS DE CÁMARA

La cámara logró capturar la imagen y el modelo realizó una predicción del número mostrado.

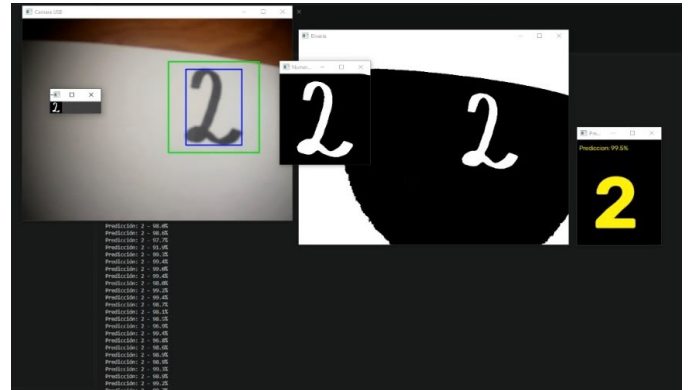


Figura 20. Resultado de la predicción con cámara USB.

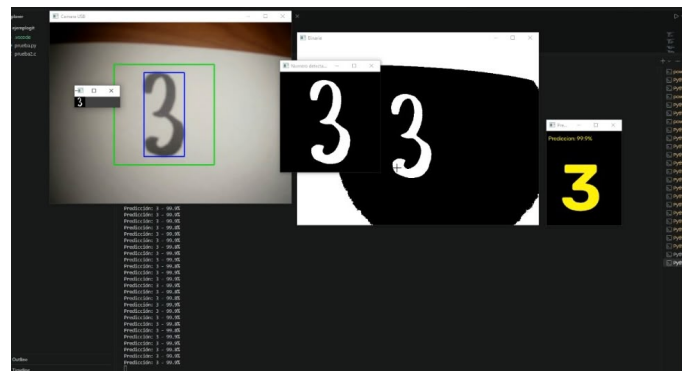


Figura 21. Resultado de la predicción con cámara USB.

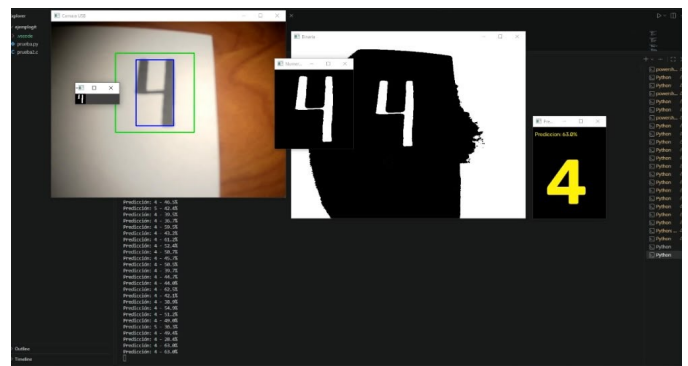


Figura 22. Resultado de la predicción con cámara USB.

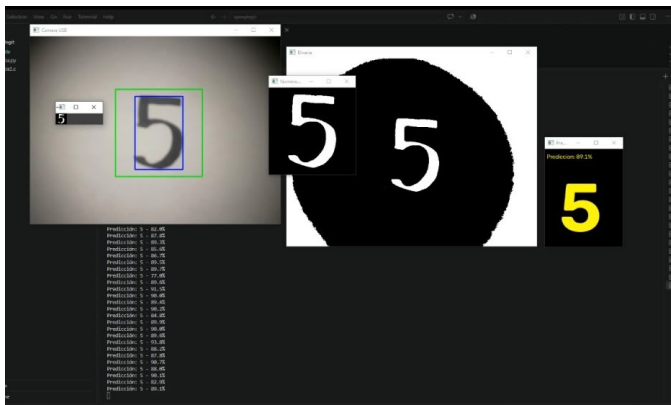


Figura 23. Resultado de la predicción con cámara USB.

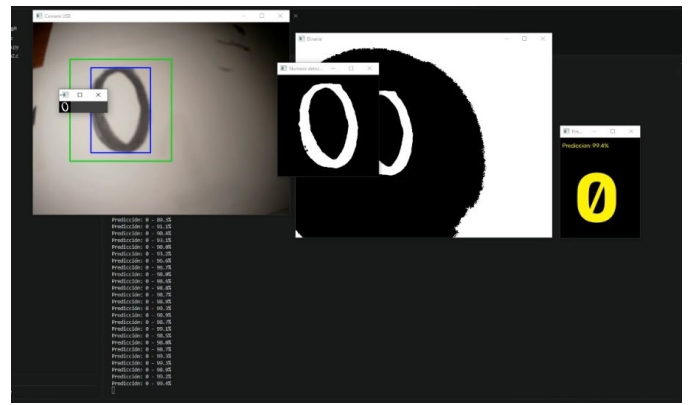


Figura 26. Resultado de la predicción con cámara USB.

VII. REPOSITORIO

El código desarrollado se encuentra disponible en el siguiente repositorio privado de GitHub:

<https://github.com/3050321780117-coder/Repopruebamaryom>

VIII. CONCLUSIONES

- Se logró entrenar correctamente la red neuronal.
- El modelo pudo reconocer números de MNIST.
- Se logró utilizar una cámara USB para realizar predicciones.

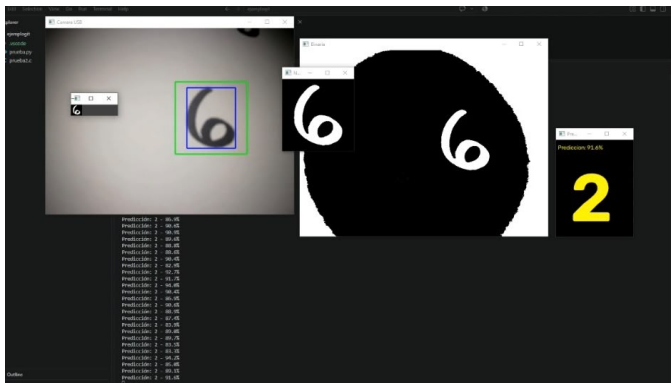


Figura 24. Resultado de la predicción con cámara USB.



Figura 25. Resultado de la predicción con cámara USB.